

Sophrosyne: Agentic Exploration of Relational Data Systems Needs Moderation

Madhav Jivrajani, Ramnatthan Alagappan, Aishwarya Ganesan

University of Illinois Urbana-Champaign

Abstract. Text2SQL agents powered by Large Language Models translate natural language intent into SQL by *exploring* the data system through tool calls before formulating the query. However, to ensure secure and scoped access, data systems construct environments with explicit API surfaces for these agents. We study and categorize the API surfaces exposed by data systems today as either *coarse-grained* or *fine-grained* and posit that choosing between the two presents a fundamental tradeoff between cost-efficient exploration and accurate SQL generation. To enable cost-efficient exploration, most data systems expose fine-grained APIs, but this inadvertently disadvantages agents: they *over-explore*, incorporating irrelevant schema elements into their query formulation and produce *inaccurate* results. We argue that curbing over-exploration is key to the effective use of these API surfaces, and propose *Sophrosyne*, a data system environment that augments API responses with *directives* that guide the agent’s exploration process. Initial results show that directives reduce over-exploration by 4.6× and boost accuracy by up to 12.4% (~4 percentage points).

1 Introduction

With advanced tool calling abilities [36], AI agents are emerging as the primary users of data systems [22, 42], and text2SQL agents in particular offer a promising approach to translating natural language queries to SQL. Agents begin by *exploring* [22] the data system’s schema to gather relevant tables, columns, and JOIN relationships prior to formulating the final query.

While exposing data systems to agents is powerful, securing and scoping access to them is crucial for production deployments [12, 26]. As a result, data systems construct environments with explicit API surfaces and authentication [6–9, 17, 18, 25, 27, 28, 32, 38, 41] using protocols such as MCP [4]. Agents execute tool calls by invoking APIs exposed to it as shown in Figure 1. Since this is becoming the ubiquitous way data systems are exposed to agents, we ask the question: *how does the exposed API surface effect the agent’s ability to perform text2SQL tasks?* To answer this question, we conduct the first study of API surfaces exposed by different data systems and find that they fall into two broad categories and differ primarily in how they enable exploration: *coarse-grained* APIs and *fine-grained* APIs. *Coarse-grained* API surfaces allow agents to retrieve the entire schema upfront for the LLM to process in its entirety and determine

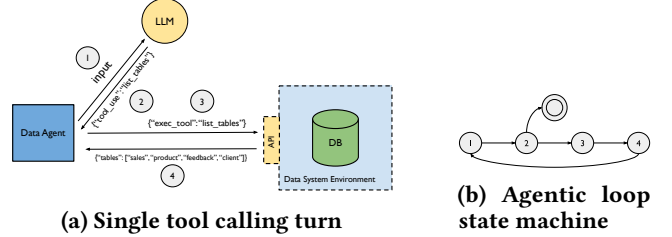


Figure 1: Agentic interaction with a data system environment

relevant elements for query formulation. *Fine-grained* API surfaces enable progressive exploration, starting with just table names and allow the agent to selectively inspect individual table schemas.

Based on our study, we observe a dichotomy between these two choices. With coarse-grained APIs, the agent has all the schema information, and has to then *distill* down what is relevant for formulating the final query. However, this leads to increased token cost (§2.2) during exploration. With fine-grained APIs, the agent pursues a more token-efficient exploration process but is not aware of the entire schema. As a result, successful query construction *necessitates* exploring all relevant tables of the database. Given that these tables are not known *a priori*, the agent inherently *over-explores* and incorporates irrelevant tables in query construction, thereby leading to inaccurate results and prohibiting effective use of these exploration APIs. Furthermore, we summarize and show in Table 1 that a majority of data systems expose fine-grained APIs, making any agent that is exposed to them prone to over-exploration and inaccurate SQL generation.

Our key insight for the effective use of fine-grained APIs is simple: let the environment provide the agent with *directives* that help constrain its future exploration. Since over-exploration constitutes exploring tables that prove to be irrelevant at a later point, we compute directives that inform the agent of potentially irrelevant ones. Our initial results show that directives reduce over-exploration by up to 4.6× and boost the accuracy of SQL generation by up to 12.4% (~4 percentage points). A key challenge here is to compute these directives, and we present *Sophrosyne*, a data system environment that achieves this.

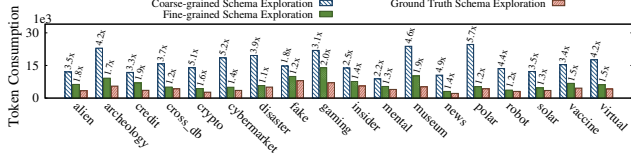
2 Motivation

We now analyze the inefficiencies of coarse-grained API surfaces and quantify the problem of over-exploration and its effects on query accuracy.

Sophrosyne is the ancient Greek virtue of prudence and moderation.

Table 1: Categorization of API Surfaces

API Surface	Data System
Coarse-grained	Google Spanner [18], PlanetScale [32]
Fine-grained	Amazon DSQL [6], Amazon MySQL [7], Amazon Postgres [8], Amazon Redshift [9], GCP BigQuery [17], Azure SQL [25], Supabase [38], Neon [28], MotherDuck DuckDB [27], Turso [41]


Figure 2: Inefficiencies of coarse-grained exploration

2.1 Setup

Our data system environment is an MCP server for SQLite [19] exposing the fine-grained API surface described in Table 2 without directives. We build our agent using OpenCode [3], a popular open-source agent implementation, and evaluate it on 157 queries from the BIRD LiveSQLBench benchmark [40] of the following difficulty composition: 25% Simple, 50% Moderate and 25% Challenging. We evaluate three models: GPT-5.4-mini, GPT-5.4, and Claude Sonnet 4.5.

2.2 Inefficiencies of Coarse-grained API Surfaces

Since token consumption directly proxies cost, we measure the inefficiency of coarse-grained API surfaces using Sonnet-4.5. With coarse-grained exploration, the agent retrieves the entire schema for every query. We estimate its token consumption for each database as the schema’s token count multiplied by the number of queries. For comparison, we compute ground truth schema token consumption by summing token counts of the tables referenced by the ground truth queries provided by BIRD, and the schema token consumption when the agent is exposed to a fine-grained API surface. Figure 2 compares these quantities per database provided by BIRD. Coarse-grained exploration consumes up to 5.7× more tokens than necessary while fine-grained exploration consumes up to 2× more making coarse-grained exploration APIs highly cost-inefficient.

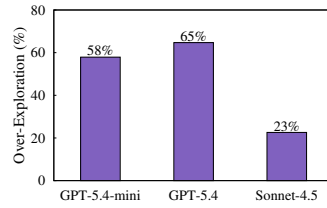
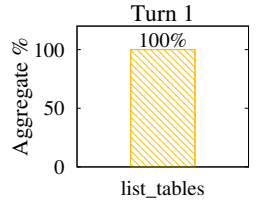
2.3 Extent of Over-exploration

To measure over-exploration, we analyze the agent’s tool call sequences. A tool call is *exploratory* if it inspects a table’s schema, i.e., a `describe_table` call or a `read_query` call of the form `PRAGMA table_info table_name`. We establish the ground truth exploration count by parsing the ground truth SQL queries provided by BIRD and extracting the number of unique tables used per query. These serve as the ground truth number of explorations needed to gather exactly the schema information required for that query.

Table 2: API surface exposed by *Sophrosyne*

API	Description
<code>list_tables</code>	Return names of tables in the database
<code>describe_table</code>	Return schema of a table
<code>get_join_info</code>	Return foreign key relationships between tables or for a specific table
<code>read_query</code>	Execute read-only queries for data exploration and error validation
<code>submit_query</code>	Submit the final SQL query

Figure 3 shows the total exploratory calls made by the agent across all queries as a percentage above the ground truth number. When exposed to fine-grained API surfaces, agents explore up to 65% more than necessary. More critically, this translates to inaccuracies in query construction as we discuss in more detail in §4.1.


Figure 3: Over-exploration in data agents

Figure 4: Turn 1 tool call composition

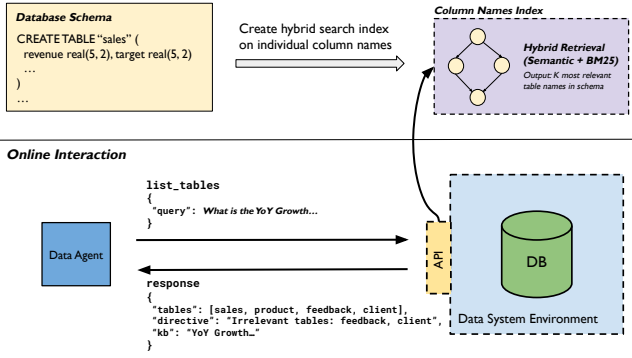
3 Design & Implementation

Over-exploration stems from exploring tables that prove to be irrelevant. Our key idea to address this issues is to compute and return *directives* that convey the set of potentially irrelevant tables as feedback to the agent to help guide its future exploration (as shown in Figure 5a). The central challenge here is computing these directives. We design and implement *Sophrosyne*, a data system environment that solves this.

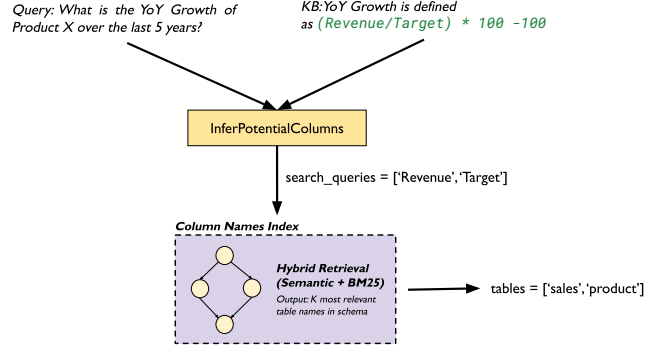
Directives are returned with the response of `list_tables`, as this is consistently the first tool call made by the agent across all models and trajectories (Figure 4). Online interaction with *Sophrosyne* requires a lightweight offline phase that constructs a hybrid search index ($\mathbf{HI}_{\text{col}}$) over individual column names, combining a simple IVF index for semantic similarity with a BM25 index [35] for keyword matches, the results of which are combined using the reciprocal rank fusion method [13]. To generate embeddings of column names for $\mathbf{HI}_{\text{col}}$, we use OpenAI’s text-embedding-3-large model. As a result, $\mathbf{HI}_{\text{col}}$ can be used to search for relevant column names and which table contains them.

The key idea behind directives is that for the databases and knowledge sources provided by the BIRD benchmark, directives are computed based on the observation that entries are defined in terms of relevant column names. Querying $\mathbf{HI}_{\text{col}}$

Lightweight Offline Phase



(a) *Sophrosyne* and the directive mechanism



(b) Computing directives from knowledge base entries

Figure 5: Overview of *Sophrosyne*'s design

using the entire knowledge base entry can lead to inaccuracies given multiple column names are likely to be present in the entry [24], we first infer potential column names by prompting a small and inexpensive model as shown in Figure 5b. The inferred column names now serve as search queries for H_{col} . Running retrieval for each of the inferred columns gives us a set of potentially *relevant* tables for this query. Directives are now the *irrelevant* tables, computed as all table names that are *not* part of this set.

Sophrosyne is implemented as an MCP server following current data systems [6–9, 17, 25, 27, 28, 38] and exposes the fine-grained API surface laid out in Table 2. We posit that directive computation should be tailored to the data system and the available context and that better mechanisms will further enhance the agent’s success. *Sophrosyne* balances offline and online costs in demonstrating how directives can be computed. Alternative approaches from schema linking [10, 39] make different cost tradeoffs but can help achieve the same goal. Furthermore, companies building data agents [2, 14, 30] can leverage the semantically richer context available to them to compute directives more effectively.

4 Evaluation

We use the setup described in §2.1. Additionally, the benchmark provides a domain-specific knowledge base to provide the agent with the knowledge needed to solve the query, and an evaluation framework to measure query accuracy. Since retrieval of knowledge entries can be inaccurate [24] and non-deterministic, we provide the agent with the exact entries needed for each query to isolate its effects and compare the following three systems on the basis of the directive mechanism: *No Directives*, *Sophrosyne*, *Sophrosyne-Oracle*. Each is implemented as an MCP server for SQLite [19].

No Directives: This is a faithful representation of existing data system environments that expose fine-grained API surfaces. It exposes the API surface laid out in Table 2 and

returns no directives with the API responses.

Sophrosyne: This also exposes the API surface laid out in Table 2, computes directives and returns them as part of the response for `list_tables` as described in §3.

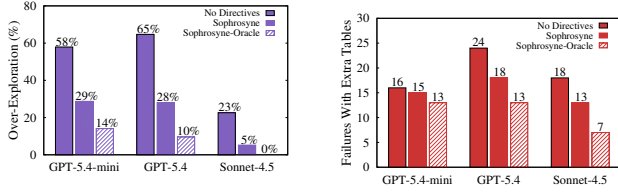
Sophrosyne-Oracle: This is identical to *Sophrosyne* except it assumes an oracle present for computing fully accurate directives. The system looks at ground truth SQL queries to compute the set of irrelevant tables thereby guaranteeing 100% accurate directives. With these, *Sophrosyne-Oracle* intends to represent the upper bound on agent performance when exposed to a fine-grained API surface.

4.1 Over-exploration & Its Effects

We measure exploration using the methodology described in §2.3. Figure 6a shows over-exploration as a percentage of exploratory tool calls above the required number. With directives, *Sophrosyne* consistently reduces over-exploration across all models, and *Sophrosyne-Oracle* reduces it further, indicating that more accurate directive computation only yields better results.

Next, we examine how directives and over-exploration affect query accuracy. Figure 6b shows the number of failed queries that reference tables absent from the ground truth. *No Directives* serves as the baseline for these failures, though not all can be attributed to over-exploration, as logical errors in query formulation are still possible. With *Sophrosyne*, these failures decrease consistently across all models. While we get improved accuracy with imperfect directives (§4.3), *Sophrosyne-Oracle* further improves it with perfect directives indicating the effect over-exploration has on query accuracy.

For Sonnet-4.5, it is worth noting that while *Sophrosyne-Oracle* completely eliminates over-exploration as seen in Figure 6a, we still see failures with extra tables for it in Figure 6b because while the agent may not explore these tables using explicit tool calls, it is still made aware of them via information returned by `get_join_info`.



(a) Measure of over-exploration (b) Failures with extra tables

Figure 6: Quantifying over-exploration and its effects

Table 3: Execution Accuracy and Token Cost

Model	System	Exec. Accuracy	Input Tokens	Output Tokens
GPT-5.4-mini	No Directives	37.6%	7.67M	217K
	Sophrosyne	39.5%	7.60M	212K
	Sophrosyne-Oracle	41.4%	6.67M	201K
GPT-5.4	No Directives	30.6%	6.28M	234K
	Sophrosyne	34.4%	6.15M	240K
	Sophrosyne-Oracle	37.6%	5.04M	233K
Sonnet 4.5	No Directives	41.4%	8.74M	343K
	Sophrosyne	42.7%	8.00M	318K
	Sophrosyne-Oracle	45.9%	6.72M	310K

4.2 Execution Accuracy & Token Cost

We use token counts from LLM providers as a proxy for inference cost [5, 31]. Table 3 summarizes execution accuracy and token counts across the three systems. *Sophrosyne* consistently improves accuracy by reducing over-exploration, and since fewer schema elements are explored, total token usage also decreases. The one exception is GPT-5.4, which incurs a 2.5% output token overhead with *Sophrosyne*; however, the reduction in input tokens offsets this increase. We discuss cost in detail in §4.3. *Sophrosyne-Oracle* consistently improves execution accuracy and improves token efficiency further indicating that better mechanisms to compute directives will only lead to better agent performance.

4.3 Directive Accuracy & System Overhead

We now discuss the accuracy and overhead of our directive mechanism (§3). The offline phase embeds column names, costing *one-tenth of a cent* totally. We treat this as negligible and ignore it in our discussion ahead. To measure accuracy of computed directives we calculate recall percentage with respect to the ground truth set of tables. The recall percentage of our directive computation mechanism across runs of the agent averages to **82.5%**.

Table 4 summarizes the base cost and overhead incurred with *Sophrosyne*. The overhead incurred is *8 cents*, covering cost to infer columns using GPT-5.4-nano and subsequently embed them (§3). *Sophrosyne* is more cost-efficient except for GPT-5.4-mini, where the base cost reduction does not absorb

Table 4: Overhead of directive computation in *Sophrosyne*

Model	System	Base Cost	Over-head	Total
GPT-5.4-mini	No Directives	\$6.73	-	\$6.73
	Sophrosyne	\$6.66	\$0.08	\$6.74
GPT-5.4	No Directives	\$19.21	-	\$19.21
	Sophrosyne	\$18.99	\$0.08	\$19.07
Sonnet-4.5	No Directives	\$31.36	-	\$31.36
	Sophrosyne	\$28.80	\$0.08	\$28.88

the overhead. Sonnet-4.5 sees the largest gains, consistent with the over-exploration reduction in Figure 6a.

5 Related Work

Text2SQL has a rich body of work ranging from building and fine-tuning custom models [16, 21, 29, 33, 34], to using LLMs with sophisticated prompting strategies [37, 39] and inference time scaling [15] to improve the accuracy of text2SQL. With LLMs becoming more adept at tool calling and interacting with data systems, works on agentic text2SQL focus on improving accuracy by synthesizing common failure modes in an offline phase and modifying the runtime workflow to provide relevant context to the agent [1, 11]. Our work is the first to study the effect of the exposed API surface on text2SQL accuracy and identify over-exploration as a symptom of an overlooked tradeoff in API design. *Sophrosyne* addresses this by augmenting API responses, and can therefore be used with any existing agent and workflow, as well as in conjunction with other works, further improving accuracy.

6 Future Work

We now outline directions for future work. Even with perfect directives, some models still over-explore, while those that avoid over-exploration can still produce failed queries with extra tables (Figure 6) suggesting the effectiveness of directives depends on the model’s instruction-following ability. Since directives are returned early, their influence may decay as the agent’s context grows due to *context rot* [20] and returning *reminders* when directives are not followed could help keep the model in line. Next, data agents answering queries spanning multiple data sources [23] need to discover these sources and subsequently relevant schema elements across them. With fine-grained surfaces, the effects of over-exploration would only be exacerbated. Directives aware of multiple sources would prove essential in this setting. Finally, in addition to guiding the exploration process, directives can also provide *performance hints* to the agent in order to construct more efficient SQL. For example, nudging the agent to use a composite index’s column prefix can dramatically improve query performance for large tables.

References

- [1] Shubham Agarwal, Asim Biswal, Sepanta Zeighami, Alvin Cheung, Joseph Gonzalez, and Aditya G. Parameswaran. Arming data agents with tribal knowledge. <https://arxiv.org/abs/2602.13521>, 2026.
- [2] Analytics at Meta. Inside meta’s home grown ai analytics agent. <https://medium.com/@AnalyticsAtMeta/inside-metas-home-grown-ai-analytics-agent-4ea6779acfb3>, March 2026.
- [3] Anomaly (SST). OpenCode: The open source AI coding agent. <https://opencode.ai>, 2025. Apache-2.0 license. Accessed: 2026-03-15.
- [4] Anthropic. Introducing the Model Context Protocol. <https://www.anthropic.com/news/model-context-protocol>, November 2024. Accessed: 2025-12-11.
- [5] Anthropic. Pricing - Claude documentation. <https://docs.claude.com/en/docs/about-claude/pricing>, 2025. Accessed: 2025-12-11.
- [6] AWS Labs. Aurora DSQL MCP server. <https://awslabs.github.io/mcp/servers/aurora-dsql-mcp-server>, 2025. Apache-2.0 license. Accessed: 2026-04-19.
- [7] AWS Labs. MySQL MCP server. <https://awslabs.github.io/mcp/servers/mysql-mcp-server>, 2025. Apache-2.0 license. Accessed: 2026-04-19.
- [8] AWS Labs. Postgres MCP server. <https://awslabs.github.io/mcp/servers/postgres-mcp-server>, 2025. Apache-2.0 license. Accessed: 2026-04-19.
- [9] AWS Labs. Redshift MCP server. <https://awslabs.github.io/mcp/servers/redshift-mcp-server>, 2025. Apache-2.0 license. Accessed: 2026-04-19.
- [10] Muhammad Imam Luthfi Balaka, David Alexander, Qiming Wang, Yue Gong, Adila Krisnadhi, and Raul Castro Fernandez. Pneuma: Leveraging LLMs for tabular data representation and retrieval in an end-to-end system. *Proc. ACM Manag. Data*, 3(3), June 2025.
- [11] Asim Biswal, Chuan Lei, Xiao Qin, Aodong Li, Balakrishnan Narayanaswamy, and Tim Kraska. AgentSM: Semantic memory for agentic text-to-SQL. <https://arxiv.org/abs/2601.15709>, 2026.
- [12] Claude Platform. Building agents that reach production systems with mcp. <https://claude.com/blog/building-agents-that-reach-production-systems-with-mcp>, April 2026.
- [13] Gordon V. Cormack, Charles L A Clarke, and Stefan Buettcher. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, SIGIR ’09, page 758–759, New York, NY, USA, 2009. Association for Computing Machinery.
- [14] Databricks. Introducing genie agent mode. <https://www.databricks.com/blog/introducing-genie-agent-mode>, April 2026.
- [15] Minghang Deng, Ashwin Ramachandran, Canwen Xu, Lanxiang Hu, Zhewei Yao, Anupam Datta, and Hao Zhang. ReFoRCE: A text-to-SQL agent with self-refinement, consensus enforcement, and column exploration. <https://arxiv.org/abs/2502.00675>, 2025.
- [16] Dawei Gao, Haibin Wang, Yaliang Li, Xiuyu Sun, Yichen Qian, Bolin Ding, and Jingren Zhou. Text-to-sql empowered by large language models: A benchmark evaluation. *Proc. VLDB Endow.*, 17(5):1132–1145, January 2024.
- [17] Google Cloud. Google cloud MCP servers: Big query. https://docs.cloud.google.com/bigquery/docs/reference/mcp/tools_overview, 2025. Preview. Accessed: 2026-03-15.
- [18] Google Cloud. Use the Spanner remote MCP server. <https://docs.cloud.google.com/spanner/docs/use-spanner-mcp>, 2026. Accessed: 2026-03-15.
- [19] Richard D. Hipp. SQLite. <https://www.sqlite.org/index.html>, 2020. Version 3.31.1.
- [20] Kelly Hong, Anton Troynikov, and Jeff Huber. Context rot: How increasing input tokens impacts LLM performance. Technical report, Chroma, July 2025.
- [21] Haoyang Li, Jing Zhang, Hanbing Liu, Ju Fan, Xiaokang Zhang, Jun Zhu, Renjie Wei, Hongyan Pan, Cuiping Li, and Hong Chen. Codes: Towards building open-source language models for text-to-sql. *Proc. ACM Manag. Data*, 2(3), May 2024.
- [22] Shu Liu, Soujanya Ponnappalli, Shreya Shankar, Sepanta Zeighami, Alan Zhu, Shubham Agarwal, Ruiqi Chen, Samion Suwito, Shuo Yuan, Ion Stoica, Matei Zaharia, Alvin Cheung, Natacha Crooks, Joseph E. Gonzalez, and Aditya G. Parameswaran. Supporting our AI overlords: Redesigning data systems to be agent-first. <https://arxiv.org/abs/2509.00997>, 2025.
- [23] Ruiying Ma, Shreya Shankar, Ruiqi Chen, Yiming Lin, Sepanta Zeighami, Rajoshi Ghosh, Abhinav Gupta, Anushrut Gupta, Tanmai Gopal, and Aditya G. Parameswaran. Can ai agents answer your data questions? a benchmark for data agents, 2026.
- [24] Xinbei Ma, Yeyun Gong, Pengcheng He, Hai Zhao, and Nan Duan. Query rewriting for retrieval-augmented large language models. <https://arxiv.org/abs/2305.14283>, 2023.
- [25] Microsoft. Azure SQL MCP Tools. <https://learn.microsoft.com/en-us/azure/data-api-builder/mcp/data-manipulation-language-tools>, 2025. Accessed: 2026-04-19.
- [26] Model Context Protocol. Building agents that reach production systems with mcp. <https://modelcontextprotocol.io/docs/tutorials/security/authorization>.
- [27] MotherDuck. MotherDuck’s DuckDB MCP server. <https://github.com/motherduckdb/mcp-server-motherduck>, 2025. MIT license. Accessed: 2026-04-19.
- [28] Neon Database. Mcp server for interacting with neon management api and databases. <https://github.com/neondatabase/mcp-server-neon>, 2026. MIT license. Accessed: 2026-03-15.
- [29] Xuan-Bang Nguyen, Xuan-Hieu Phan, and Massimo Piccardi. Fine-tuning text-to-sql models with reinforcement-learning training objectives. *Natural Language Processing Journal*, 10:100135, 2025.
- [30] OpenAI. Inside our in-house data agent. <https://openai.com/index/inside-our-in-house-data-agent/>, January 2026.
- [31] OpenAI. OpenAI API pricing. <https://developers.openai.com/api/docs/pricing>, 2026. Accessed: 2026-04-02.
- [32] PlanetScale. PlanetScale MCP server. <https://github.com/planetscale/mcp-server>, 2026. Apache-2.0 license. Accessed: 2026-03-15.
- [33] Mohammadreza Pourreza and Davood Rafiei. DTS-SQL: Decomposed text-to-SQL with small large language models. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen, editors, *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 8212–8220, Miami, Florida, USA, November 2024. Association for Computational Linguistics.
- [34] Yang Qin, Chao Chen, Zhihang Fu, Ze Chen, Dezhong Peng, Peng Hu, and Jieping Ye. ROUTE: Robust multitask tuning and collaboration for text-to-SQL. In *The Thirteenth International Conference on Learning Representations*, 2025.
- [35] Stephen E. Robertson and Steve Walker. Some simple effective approximations to the 2-poisson model for probabilistic weighted retrieval. In *Proceedings of the 17th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 232–241,

- Dublin, Ireland, 1994. Springer-Verlag.
- [36] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. <https://arxiv.org/abs/2302.04761>, 2023.
 - [37] Vladislav Shkapenyuk, Divesh Srivastava, Theodore Johnson, and Parisa Ghane. Automatic metadata extraction for text-to-SQL. <https://arxiv.org/abs/2505.19988>, 2025.
 - [38] Supabase Community. Supabase MCP server. <https://github.com/supabase-community/supabase-mcp>, 2025. Apache-2.0 license. Accessed: 2026-03-15.
 - [39] Shayan Talaei, Mohammadreza Pourreza, Yu-Chen Chang, Azalia Mirhoseini, and Amin Saberi. CHES: Contextual harnessing for efficient SQL synthesis. <https://arxiv.org/abs/2405.16755>, 2024.
 - [40] BIRD Team. LiveSQLBench: A dynamic and contamination-free benchmark for evaluating LLMs on real-world text-to-SQL tasks. <https://github.com/bird-bench/livesqlbench>, 2025. Accessed: 2025-05-22.
 - [41] Turso. Introducing the Turso database MCP. <https://turso.tech/blog/introducing-the-turso-database-mcp-server>, 2025. Accessed: 2026-05-03.
 - [42] Matei Zaharia. Bridging the operational and analytical worlds with Lakebase. In *Proceedings of the 51st International Conference on Very Large Data Bases, VLDB '25*, 2025. Keynote 3.